Математическое моделирование

Лабораторная работа № 6

Элсаиед Адел

Содержание

1 Цель работы **6**

2 Задание **7**

3 Выполнение лабораторной работы **8**
3.1 Теоретические сведения 8

4 Эпидемиологическая модель SIR: численное решение и визуализация **10**
4.1 Инициализация проекта и загрузка пакетов 10
4.2 Начальные условия и временной интервал 11
4.3 Параметры модели 11
4.4 Первая модель 11
4.5 Вторая модель 12
4.6 Утилита: один прогон модели 12
4.7 Запуск первой модели 15
4.8 Запуск второй модели 15
4.9 Вывод первых строк таблиц 16
4.10 Показать график в интерактивной среде 16

5 Параметрическое исследование эпидемиологической модели SIR **17**
5.1 Активация проекта и загрузка пакетов 17
5.2 Установка каталогов 17
5.3 Определение моделей 18
5.4 Базовые параметры 18
5.5 Функция для запуска одного эксперимента 19
5.6 Запуск базового эксперимента: первая система 22
5.7 Визуализация базового эксперимента 23
5.8 Фазовый портрет I(S) 24
5.9 Второй базовый эксперимент 25
5.10 Параметрическое сканирование 27
5.11 Запуск всех экспериментов 28
5.12 Анализ результатов сканирования 30
5.13 Сравнительный график S(t) 31
5.14 Сравнительный график I(t) 32
5.15 Сравнительный график R(t) 33
5.16 Сравнительный фазовый портрет S-I 35

5.17 График зависимости I_max от параметров 36
5.18 График зависимости norm_final от параметров 37
5.19 Бенчмаркинг 39
5.20 График времени вычисления 41
5.21 Сохранение всех результатов 42
5.22 Базовые эксперименты 43
5.23 Параметрическое сканирование 47
5.24 Выводы 54

Список литературы **55**

Список иллюстраций

5.1 Первая модель: зависимость S(t), I(t), R(t) 43
5.2 Первая модель: фазовый портрет 44
5.3 Вторая модель: зависимость S(t), I(t), R(t) 45
5.4 Вторая модель: фазовый портрет 46
5.5 Сканирование траекторий S(t) 47
5.6 Сканирование траекторий I(t) 48
5.7 Сканирование траекторий R(t) 49
5.8 Сканирование фазовых траекторий 50
5.9 Зависимость norm_final 51
5.10 Зависимость I_{max} 52
5.11 Зависимость времени вычисления 53

Список таблиц

1 Цель работы

Изучить модель эпидемии SIR

2 Задание

1. Изучить модель эпидемии
2. Построить графики изменения числа особей в каждой из трех групп. Рассмотреть, как будет протекать эпидемия в случае: $I(0) \leq I^*$, $I(0) > I^*$

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Рассмотрим простейшую модель эпидемии. Предположим, что некая популяция, состоящая из N особей, (считаем, что популяция изолирована) подразделяется на три группы. Первая группа - это восприимчивые к болезни, но пока здоровые особи, обозначим их через $S(t)$. Вторая группа – это число инфицированных особей, которые также при этом являются распространителями инфекции, обозначим их $I(t)$. А третья группа, обозначающаяся через $R(t)$ – это здоровые особи с иммунитетом к болезни. До того, как число заболевших не превышает критического значения I^*, считаем, что все больные изолированы и не заражают здоровых. Когда $I(t) > I^*$, тогда инфицирование способны заражать восприимчивых к болезни особей.

Таким образом, скорость изменения числа $S(t)$ меняется по следующему закону:

$$\frac{dS}{dt} = \begin{cases} -\alpha S & \text{,если } I(t) > I^* \\ 0 & \text{,если } I(t) \leq I^* \end{cases}$$

Поскольку каждая восприимчивая к болезни особь, которая, в конце концов, заболевает, сама становится инфекционной, то скорость изменения числа инфекционных особей представляет разность за единицу времени между заразившимися и теми, кто уже болеет и лечится. Т.е.:

$$\frac{dI}{dt} = \begin{cases} \alpha S - \beta I & \text{,если } I(t) > I^* \\ -\beta I & \text{,если } I(t) \leq I^* \end{cases}$$

А скорость изменения выздоравливающих особей (при этом приобретающие иммунитет к болезни):

$$\frac{dR}{dt} = \beta I$$

Постоянные пропорциональности α, β - это коэффициенты заболеваемости и выздоровления соответственно. Для того, чтобы решения соответствующих уравнений определялось однозначно, необходимо задать начальные условия. Считаем, что на начало эпидемии в момент времени $t = 0$ нет особей с иммунитетом к болезни $R(0) = 0$, а число инфицированных и восприимчивых к болезни особей $I(0)$ и $S(0)$ соответственно. Для анализа картины протекания эпидемии необходимо рассмотреть два случая: $I(0) \leq I^*$ и $I(0) > I^*$

3.1.1 Задача

На одном острове вспыхнула эпидемия. Известно, что из всех проживающих на острове $N = 11400$ в момент начала эпидемии ($t = 0$) число заболевших людей (являющихся распространителями инфекции) $I(0) = 250$, А число здоровых людей с иммунитетом к болезни $R(0) = 47$. Таким образом, число людей восприимчивых к болезни, но пока здоровых, в начальный момент времени $S(0) = N - I(0) - R(0)$. Постройте графики изменения числа особей в каждой из трех групп.

Рассмотрите, как будет протекать эпидемия в случае: 1. $I(0) \leq I^*$ 2. $I(0) > I^*$

Для моделирования процесса и построения графиков использовались внешние файлы с программным кодом:

4 Эпидемиологическая модель SIR: численное решение и визуализация

Цель: решить две системы ОДУ для модели распространения заболевания, построить графики S(t), I(t), R(t), сохранить изображения и таблицы с результатами.

4.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using CSV
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.2 Начальные условия и временной интервал

```
N = 11400.0
I0 = 250.0
R0 = 47.0
S0 = N - I0 - R0

u0 = [S0, I0, R0]

t0 = 0.0
tmax = 200.0
tspan = (t0, tmax)
```

4.3 Параметры модели

```
a = 0.11
b = 0.02

p = (a = a, b = b)
```

4.4 Первая модель

S(t) — число восприимчивых I(t) — число инфицированных R(t) — число выбывших / выздоровевших

dS/dt = 0 dI/dt = b*I dR/dt = -b*I

```
function sir_case_1!(dy, y, p, t)
    dy[1] = 0.0
    dy[2] = p.b * y[2]
    dy[3] = -p.b * y[2]
end
```

4.5 Вторая модель

dS/dt = -aS $dI/dt = a$S - bI $dR/dt = b$I

```
function sir_case_2!(dy, y, p, t)
    dy[1] = -p.a * y[1]
    dy[2] =  p.a * y[1] - p.b * y[2]
    dy[3] =  p.b * y[2]
end
```

4.6 Утилита: один прогон модели

```
function run_case(case_name; model!, u0, tspan, p, nt = 1000)
```

— Сетка по времени —

```
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
    prob = ODEProblem(model!, u0, tspan, p)
    sol = solve(prob, Tsit5(), saveat = tgrid)
```

— Таблица с результатами —

```
df = DataFrame(
    t = sol.t,
    S = [u[1] for u in sol.u],
    I = [u[2] for u in sol.u],
    R = [u[3] for u in sol.u]
)
```

— График S(t), I(t), R(t) —

```
plt_time = plot(
    sol,
    xlabel = "t",
    ylabel = "Численность",
    label = ["S(t) — восприимчивые" "I(t) — инфицированные" "R(t) — выбывшие"],
    lw = 2,
    title = "Динамика SIR — $case_name",
    legend = :topright
)
```

— Фазовый портрет I(S) —

```
plt_phase_si = plot(
    sol,
    idxs = (1, 2),
    xlabel = "S",
    ylabel = "I",
    lw = 2,
    label = "Фазовая траектория I(S)",
    title = "Фазовый портрет S-I — $case_name",
```

```
        legend = :topright
    )
```

— Фазовый портрет R(I) —

```
    plt_phase_ir = plot(
        sol,
        idxs = (2, 3),
        xlabel = "I",
        ylabel = "R",
        lw = 2,
        label = "Фазовая траектория R(I)",
        title = "Фазовый портрет I-R — $case_name",
        legend = :topright
    )
```

— Сохранение графиков —

```
    savefig(plt_time, plotsdir(script_name, "$(case_name)_time.png"))
    savefig(plt_phase_si, plotsdir(script_name, "$(case_name)_phase_SI.png"))
    savefig(plt_phase_ir, plotsdir(script_name, "$(case_name)_phase_IR.png"))
```

— Сохранение таблицы —

```
    CSV.write(datadir(script_name, "table_$(case_name).csv"), df)
```

— Сохранение данных в JLD2 —

```
    @save datadir(script_name, "data_$(case_name).jld2") df p u0 tspan nt

    return (
```

```
        sol = sol,
        df = df,
        plt_time = plt_time,
        plt_phase_si = plt_phase_si,
        plt_phase_ir = plt_phase_ir
    )
end
```

4.7 Запуск первой модели

```
res_1 = run_case(
    "sir_case_1";
    model! = sir_case_1!,
    u0 = u0,
    tspan = tspan,
    p = p,
    nt = 1000
)
```

4.8 Запуск второй модели

```
res_2 = run_case(
    "sir_case_2";
    model! = sir_case_2!,
    u0 = u0,
    tspan = tspan,
    p = p,
```

```
    nt = 1000
)
```

4.9 Вывод первых строк таблиц

```
println("Первые 5 строк таблицы результатов для первой модели:")
println(first(res_1.df, 5))

println()
println("Первые 5 строк таблицы результатов для второй модели:")
println(first(res_2.df, 5))
```

4.10 Показать график в интерактивной среде

```
res_1.plt_time
```

5 Параметрическое исследование эпидемиологической модели SIR

5.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
using CSV
using Statistics
```

5.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.3 Определение моделей

Первая система: dS/dt = 0 dI/dt = b*I dR/dt = -b*I

```
function sir_case_1!(dy, y, p, t)
    dy[1] = 0.0
    dy[2] = p.b * y[2]
    dy[3] = -p.b * y[2]
end
```

Вторая система: dS/dt = -a*S dI/dt = a*S - b*I dR/dt = b*I

```
function sir_case_2!(dy, y, p, t)
    dy[1] = -p.a * y[1]
    dy[2] =  p.a * y[1] - p.b * y[2]
    dy[3] =  p.b * y[2]
end
```

5.4 Базовые параметры

```
base_params = Dict(
    :N => 11400.0,
    :I0 => 250.0,
    :R0 => 47.0,

    :tspan => (0.0, 200.0),
    :nt => 1000,

    :model_type => :case1,     # :case1 или :case2
```

```
    :a => 0.11,
    :b => 0.02,

    :solver => Tsit5(),
    :experiment_name => "sir_base_experiment"
)

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

5.5 Функция для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    N = params[:N]
    I0 = params[:I0]
    R0 = params[:R0]
    S0 = N - I0 - R0

    tspan = params[:tspan]
    nt = params[:nt]
    model_type = params[:model_type]

    a = params[:a]
    b = params[:b]
```

```
    solver = params[:solver]

    u0 = [S0, I0, R0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    p = (a = a, b = b)

    if model_type == :case1
        prob = ODEProblem(sir_case_1!, u0, tspan, p)
    else
        prob = ODEProblem(sir_case_2!, u0, tspan, p)
    end

    sol = solve(prob, solver; saveat = tgrid)

    S_vals = [u[1] for u in sol.u]
    I_vals = [u[2] for u in sol.u]
    R_vals = [u[3] for u in sol.u]
```

— Мини-анализ —

```
    S_final = S_vals[end]
    I_final = I_vals[end]
    R_final = R_vals[end]

    S_max = maximum(S_vals)
    I_max = maximum(I_vals)
    R_max = maximum(R_vals)
```

```
    S_min = minimum(S_vals)
    I_min = minimum(I_vals)
    R_min = minimum(R_vals)

    S_mean = mean(S_vals)
    I_mean = mean(I_vals)
    R_mean = mean(R_vals)

    total_final = S_final + I_final + R_final
    norm_final = sqrt(S_final^2 + I_final^2 + R_final^2)

    return Dict(
        "solution" => sol,
        "t_points" => sol.t,

        "S_values" => S_vals,
        "I_values" => I_vals,
        "R_values" => R_vals,

        "S_final" => S_final,
        "I_final" => I_final,
        "R_final" => R_final,

        "S_max" => S_max,
        "I_max" => I_max,
        "R_max" => R_max,

        "S_min" => S_min,
```

```
        "I_min" => I_min,
        "R_min" => R_min,

        "S_mean" => S_mean,
        "I_mean" => I_mean,
        "R_mean" => R_mean,

        "total_final" => total_final,
        "norm_final" => norm_final,

        "parameters" => params
    )
end
```

5.6 Запуск базового эксперимента: первая система

```
data_base, path_base = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "sir",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" S_final: ", data_base["S_final"])
```

```
println(" I_final: ", data_base["I_final"])
println(" R_final: ", data_base["R_final"])
println(" I_max: ", data_base["I_max"])
println(" total_final: ", data_base["total_final"])
println(" norm_final: ", round(data_base["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base)
```

5.7 Визуализация базового эксперимента

```
p1 = plot(
    data_base["t_points"], data_base["S_values"],
    label = "S(t) — восприимчивые",
    xlabel = "t",
    ylabel = "Численность",
    title = "Базовый эксперимент: model_type=$(base_params[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

plot!(
    p1,
    data_base["t_points"], data_base["I_values"],
    label = "I(t) — инфицированные",
    lw = 2
)
```

```
plot!(
    p1,
    data_base["t_points"], data_base["R_values"],
    label = "R(t) — выбывшие",
    lw = 2
)

savefig(p1, plotsdir(script_name, "single_experiment_case1.png"))
```

5.8 Фазовый портрет I(S)

```
p2 = plot(
    data_base["S_values"], data_base["I_values"],
    xlabel = "S",
    ylabel = "I",
    label = "Фазовая траектория I(S)",
    title = "Фазовый портрет S-I: model_type=$(base_params[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p2, plotsdir(script_name, "single_experiment_case1_phase_SI.png"))
```

5.9 Второй базовый эксперимент

```
base_params2 = copy(base_params)
base_params2[:model_type] = :case2
base_params2[:experiment_name] = "sir_base_experiment_case2"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),
    base_params2,
    run_single_experiment;
    prefix = "sir",
    tag = false,
    verbose = true
)

println("\nРезультаты второго базового эксперимента:")
println(" S_final: ", data_base2["S_final"])
println(" I_final: ", data_base2["I_final"])
println(" R_final: ", data_base2["R_final"])
println(" I_max: ", data_base2["I_max"])
println(" total_final: ", data_base2["total_final"])
println(" norm_final: ", round(data_base2["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base2)

p3 = plot(
    data_base2["t_points"], data_base2["S_values"],
    label = "S(t) — восприимчивые",
    xlabel = "t",
```

```
    ylabel = "Численность",
    title = "Базовый эксперимент: model_type=$(base_params2[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

plot!(
    p3,
    data_base2["t_points"], data_base2["I_values"],
    label = "I(t) — инфицированные",
    lw = 2
)

plot!(
    p3,
    data_base2["t_points"], data_base2["R_values"],
    label = "R(t) — выбывшие",
    lw = 2
)

savefig(p3, plotsdir(script_name, "single_experiment_case2.png"))

p4 = plot(
    data_base2["S_values"], data_base2["I_values"],
    xlabel = "S",
    ylabel = "I",
    label = "Фазовая траектория I(S)",
```

```
    title = "Фазовый портрет S-I: model_type=$(base_params2[:model_type])",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p4, plotsdir(script_name, "single_experiment_case2_phase_SI.png"))
```

5.10 Параметрическое сканирование

```
param_grid = Dict(
    :N => [11400.0],
    :I0 => [250.0],
    :R0 => [47.0],

    :tspan => [(0.0, 200.0)],
    :nt => [1000],

    :model_type => [:case1, :case2],

    :a => [0.05, 0.08, 0.11, 0.15],
    :b => [0.01, 0.02, 0.03, 0.05],

    :solver => [Tsit5()],
    :experiment_name => ["sir_parametric_scan"]
)
```

```
all_params = dict_list(param_grid)

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые a: ", param_grid[:a])
println("Исследуемые b: ", param_grid[:b])
println("="^60)
```

5.11 Запуск всех экспериментов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println(
        "Прогресс: $i/$(length(all_params)) | ",
        "model_type=$(params[:model_type]) | ",
        "a=$(params[:a]) | b=$(params[:b])"
    )

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
```

```
        tag = false,
        verbose = false
    )

    result_summary = merge(
        params,
        Dict(
            :S_final => data["S_final"],
            :I_final => data["I_final"],
            :R_final => data["R_final"],

            :S_max => data["S_max"],
            :I_max => data["I_max"],
            :R_max => data["R_max"],

            :S_min => data["S_min"],
            :I_min => data["I_min"],
            :R_min => data["R_min"],

            :S_mean => data["S_mean"],
            :I_mean => data["I_mean"],
            :R_mean => data["R_mean"],

            :total_final => data["total_final"],
            :norm_final => data["norm_final"],
            :filepath => path
        )
    )
```

```
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        S = data["S_values"],
        I = data["I_values"],
        R = data["R_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        b = fill(params[:b], length(data["t_points"]))
    )

    push!(all_dfs, df)
end
```

5.12 Анализ результатов сканирования

```
results_df = DataFrame(all_results)
full_timeseries_df = vcat(all_dfs...)

println("\nСводная таблица результатов:")
println(first(results_df, 10))

CSV.write(datadir(script_name, "results_summary.csv"), results_df)
CSV.write(datadir(script_name, "timeseries_full.csv"), full_timeseries_df)
```

5.13 Сравнительный график S(t)

```
p5 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p5,
        data["t_points"],
        data["S_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p5,
```

```
    xlabel = "t",
    ylabel = "S(t)",
    title = "Сканирование: траектории S(t)",
    legend = :outerright,
    grid = true
)

savefig(p5, plotsdir(script_name, "parametric_scan_S_comparison.png"))
```

5.14 Сравнительный график I(t)

```
p6 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p6,
```

```
        data["t_points"],
        data["I_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p6,
    xlabel = "t",
    ylabel = "I(t)",
    title = "Сканирование: траектории I(t)",
    legend = :outerright,
    grid = true
)

savefig(p6, plotsdir(script_name, "parametric_scan_I_comparison.png"))
```

5.15 Сравнительный график R(t)

```
p7 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
```

```
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p7,
        data["t_points"],
        data["R_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p7,
    xlabel = "t",
    ylabel = "R(t)",
    title = "Сканирование: траектории R(t)",
    legend = :outerright,
    grid = true
)

savefig(p7, plotsdir(script_name, "parametric_scan_R_comparison.png"))
```

5.16 Сравнительный фазовый портрет S-I

```
p8 = plot(size = (950, 520), dpi = 150)

for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "$(params[:model_type]), a=$(params[:a]), b=$(params[:b])"

    plot!(
        p8,
        data["S_values"],
        data["I_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    p8,
```

```
    xlabel = "S",
    ylabel = "I",
    title = "Сканирование: фазовые траектории S-I",
    legend = :outerright,
    grid = true
)

savefig(p8, plotsdir(script_name, "parametric_scan_phase_SI_comparison.png"))
```

5.17 График зависимости I_max от параметров

```
p9 = plot(size = (900, 520), dpi = 150)

case1_sub = results_df[results_df.model_type .== :case1, :]
case2_sub = results_df[results_df.model_type .== :case2, :]

if nrow(case1_sub) > 0
    plot!(
        p9,
        case1_sub.b,
        case1_sub.I_max,
        seriestype = :scatter,
        label = "case1: I_max от b"
    )
end

if nrow(case2_sub) > 0
```

```
    plot!(
        p9,
        case2_sub.a,
        case2_sub.I_max,
        seriestype = :scatter,
        label = "case2: I_max от a"
    )
end

plot!(
    p9,
    xlabel = "Параметр модели",
    ylabel = "I_max",
    title = "Зависимость максимального числа инфицированных от параметров",
    legend = :topleft,
    grid = true
)

savefig(p9, plotsdir(script_name, "I_max_vs_parameter.png"))
```

5.18 График зависимости norm_final от параметров

```
p10 = plot(size = (900, 520), dpi = 150)

if nrow(case1_sub) > 0
    plot!(
        p10,
```

```
        case1_sub.b,
        case1_sub.norm_final,
        seriestype = :scatter,
        label = "case1: norm_final от b"
    )
end

if nrow(case2_sub) > 0
    plot!(
        p10,
        case2_sub.a,
        case2_sub.norm_final,
        seriestype = :scatter,
        label = "case2: norm_final от a"
    )
end

plot!(
    p10,
    xlabel = "Параметр модели",
    ylabel = "norm_final",
    title = "Зависимость norm_final от параметров модели",
    legend = :topleft,
    grid = true
)

savefig(p10, plotsdir(script_name, "norm_final_vs_parameter.png"))
```

5.19 Бенчмаркинг

```
println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        N = params[:N]
        I0 = params[:I0]
        R0 = params[:R0]
        S0 = N - I0 - R0

        u0 = [S0, I0, R0]
        p = (a = params[:a], b = params[:b])

        if params[:model_type] == :case1
            prob = ODEProblem(sir_case_1!, u0, params[:tspan], p)
        else
            prob = ODEProblem(sir_case_2!, u0, params[:tspan], p)
        end

        return solve(
            prob,
            params[:solver];
            saveat = LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
```

```
        )
    end

    println(
        "\nБенчмарк для model_type=$(params[:model_type]), ",
        "a=$(params[:a]), b=$(params[:b]):"
    )

    bmark = @benchmark $benchmark_run() samples = 80 evals = 1
    tsec = median(bmark).time / 1e9

    println(" Медианное время: ", round(tsec; digits = 6), " сек")

    push!(
        benchmark_results,
        (
            model_type = string(params[:model_type]),
            a = params[:a],
            b = params[:b],
            time = tsec
        )
    )
end

bench_df = DataFrame(benchmark_results)
CSV.write(datadir(script_name, "benchmark_results.csv"), bench_df)
```

5.20 График времени вычисления

```
p11 = plot(size = (900, 520), dpi = 150)

bench_case1 = bench_df[bench_df.model_type .== "case1", :]
bench_case2 = bench_df[bench_df.model_type .== "case2", :]

if nrow(bench_case1) > 0
    plot!(
        p11,
        bench_case1.b,
        bench_case1.time,
        seriestype = :scatter,
        label = "case1: время от b"
    )
end

if nrow(bench_case2) > 0
    plot!(
        p11,
        bench_case2.a,
        bench_case2.time,
        seriestype = :scatter,
        label = "case2: время от a"
    )
end

plot!(
```

```
    p11,
    xlabel = "Параметр модели",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметров",
    legend = :topleft,
    grid = true
)

savefig(p11, plotsdir(script_name, "computation_time_vs_parameter.png"))
```

5.21 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params base_params2 param_grid al
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5 p6 p7 p8 p9 p10 p11

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)

println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовые эксперименты")
println(" • data/$(script_name)/parametric_scan/ - параметрическое сканирование")
println(" • data/$(script_name)/results_summary.csv - сводная таблица")
println(" • data/$(script_name)/timeseries_full.csv - полные временные ряды")
println(" • data/$(script_name)/benchmark_results.csv - результаты бенчмаркинга")
println(" • data/$(script_name)/all_results.jld2 - сводные данные")
println(" • plots/$(script_name)/ - все графики")
```

```
println(" • data/$(script_name)/all_plots.jld2 - объекты графиков")
```

5.22 Базовые эксперименты

5.22.1 Первая модель (model_type = case1)

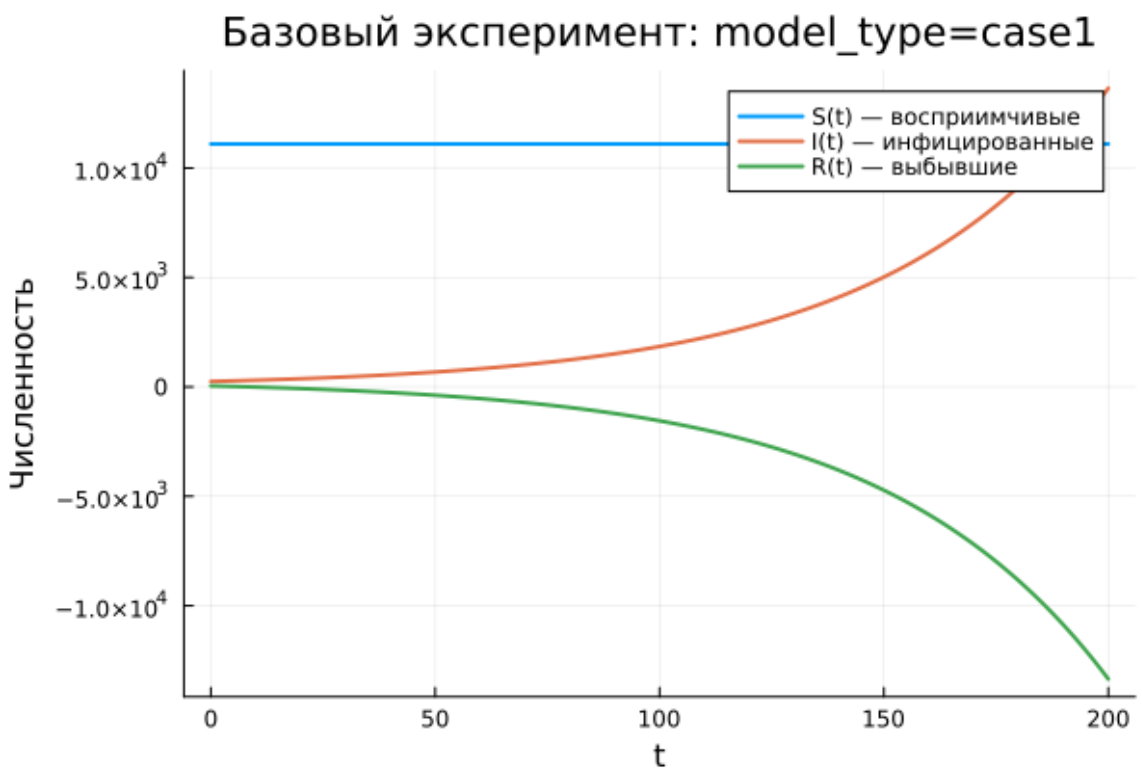

Рисунок 5.1: Первая модель: зависимость S(t), I(t), R(t)

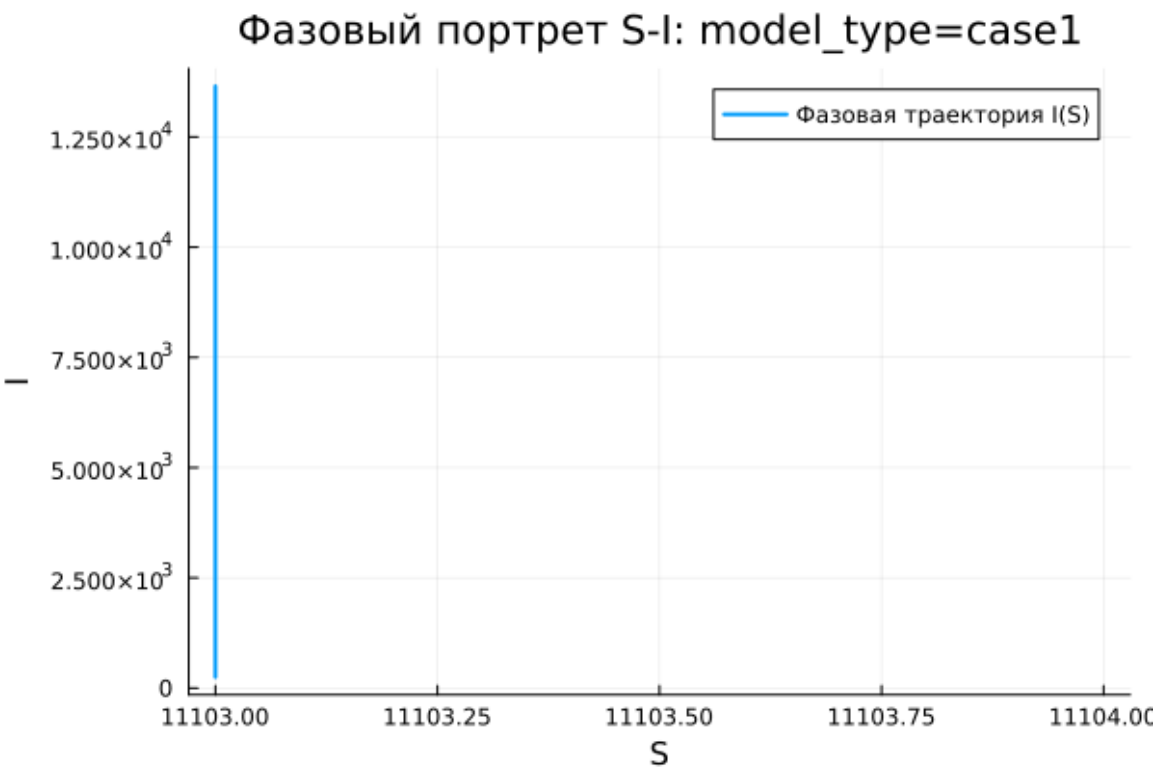

Рисунок 5.2: Первая модель: фазовый портрет

Для первой модели наблюдается нетипичное поведение системы. Число восприимчивых $S(t)$ остаётся постоянным во времени, что соответствует уравнению $dS/dt = 0$. При этом число инфицированных $I(t)$ экспоненциально возрастает, а число выбывших $R(t)$ убывает и становится отрицательным.

Такое поведение не соответствует классической эпидемиологической модели и связано с особенностями заданной системы уравнений. В частности, отсутствует механизм ограничения роста инфицированных, а также нарушается физический смысл переменной $R(t)$, которая должна быть неотрицательной.

Фазовый портрет подтверждает данный вывод. Траектория практически вырождается в вертикальную линию, поскольку переменная S остаётся постоянной, а изменение происходит только по оси I. Это означает, что система фактически редуцируется к одномерной динамике.

Следовательно, первая модель демонстрирует неограниченный рост инфи-

цированных без стабилизации и не может рассматриваться как адекватная модель распространения заболевания.

5.22.2 Вторая модель (model_type = case2)

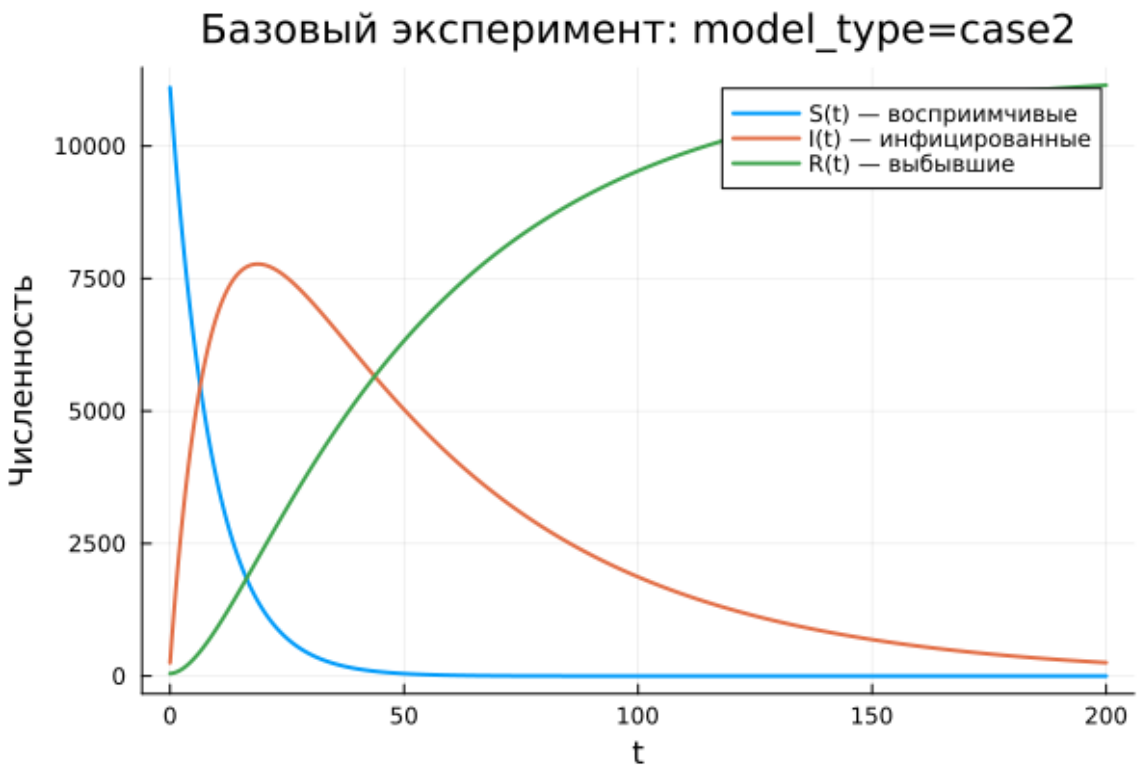

Рисунок 5.3: Вторая модель: зависимость S(t), I(t), R(t)

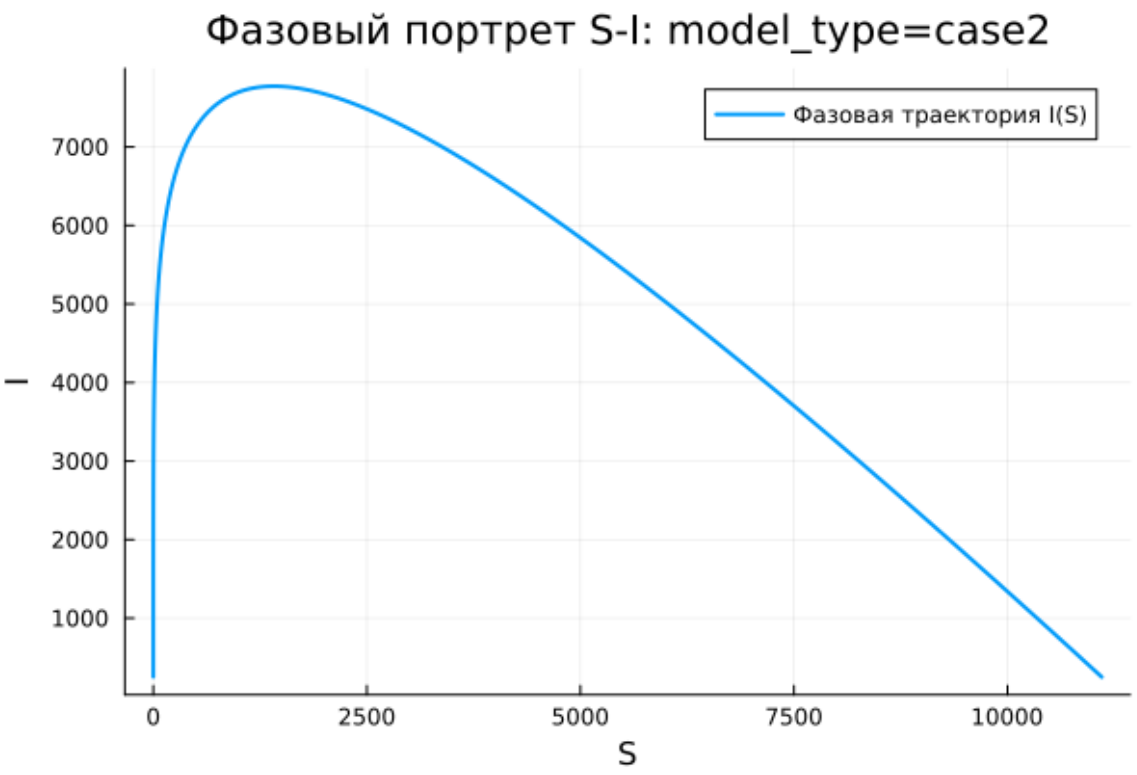

Рисунок 5.4: Вторая модель: фазовый портрет

Во второй модели наблюдается типичное поведение SIR-системы. Число восприимчивых $S(t)$ монотонно убывает, что отражает процесс заражения. Число инфицированных $I(t)$ сначала возрастает, достигает максимума, а затем постепенно уменьшается, стремясь к нулю. Число выбывших $R(t)$, напротив, монотонно возрастает.

Такое поведение соответствует эпидемическому процессу с конечным числом населения. В начальный момент происходит активное распространение инфекции, затем по мере уменьшения числа восприимчивых скорость заражения падает, и эпидемия затухает.

Фазовый портрет имеет вид незамкнутой кривой, которая сначала поднимается вверх (рост I при уменьшении S), затем плавно спускается вниз. Это отражает переход системы через максимум числа инфицированных и дальнейшее затухание эпидемии.

В отличие от первой модели, здесь наблюдается устойчивое стремление к стационарному состоянию: $I(t) \to 0$, $S(t)$ достигает некоторого остаточного уровня, а $R(t)$ фиксируется на конечном значении.

5.23 Параметрическое сканирование

5.23.1 Траектории $S(t)$ для различных параметров

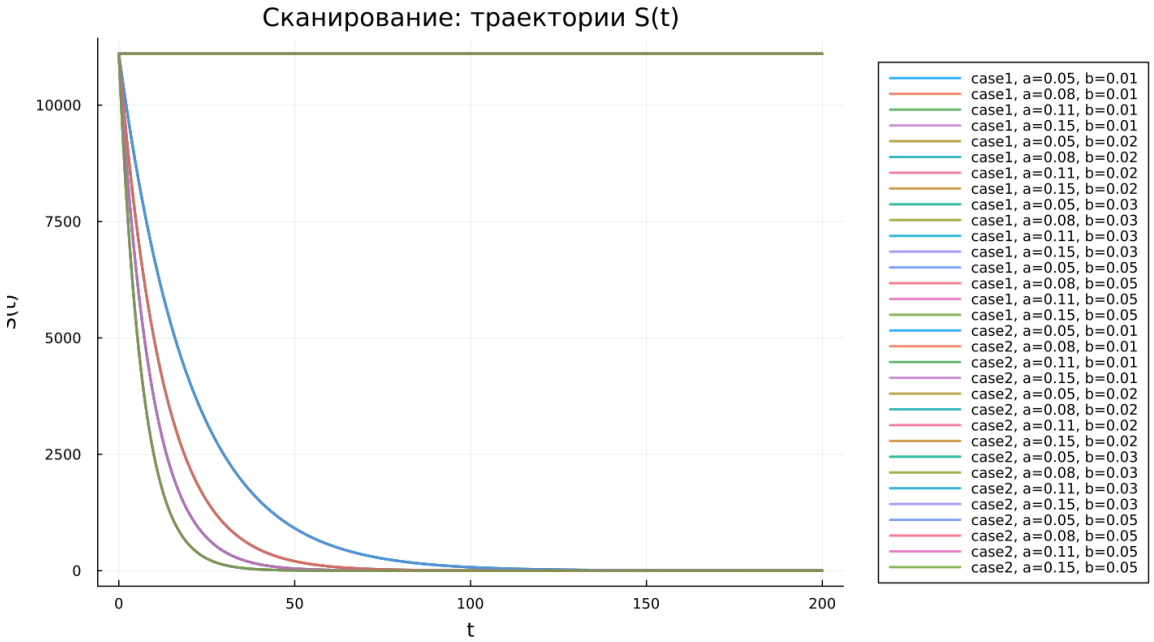

Рисунок 5.5: Сканирование траекторий S(t)

Анализ траекторий $S(t)$ показывает различное поведение моделей при изменении параметров. Для первой модели (case1) значение $S(t)$ остаётся постоянным при любых параметрах, что связано с уравнением $dS/dt = 0$. Таким образом, параметрическое изменение коэффициентов не влияет на динамику восприимчивых.

Во второй модели (case2) наблюдается экспоненциальное убывание $S(t)$, причём скорость уменьшения существенно зависит от параметра a. При увеличе-

нии a восприимчивые уменьшаются быстрее, что соответствует ускорению процесса заражения.

Основные наблюдения:

- в первой модели $S(t)$ не изменяется;
- во второй модели $S(t)$ убывает;
- параметр a определяет скорость убывания восприимчивых.

5.23.2 Траектории $I(t)$ для различных параметров

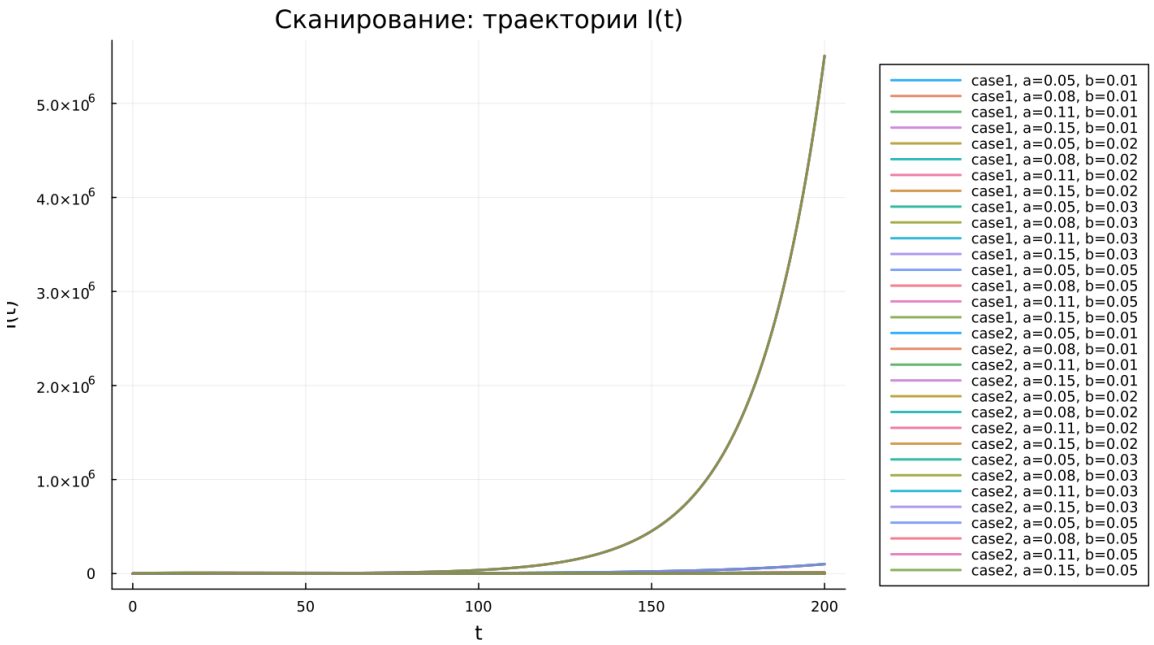

Рисунок 5.6: Сканирование траекторий I(t)

Для первой модели наблюдается экспоненциальный рост $I(t)$ для всех значений параметров. При увеличении параметра b скорость роста существенно возрастает, что приводит к крайне большим значениям инфицированных.

Во второй модели динамика существенно отличается. Число инфицированных сначала увеличивается, достигая максимума, а затем уменьшается. Изменение параметра a влияет на высоту пика и скорость его достижения: при

больших значениях a пик достигается быстрее, но также быстрее происходит спад.

- первая модель демонстрирует неограниченный рост инфицированных;
- вторая модель демонстрирует типичную эпидемическую волну;
- параметры управляют скоростью и масштабом распространения инфекции.

5.23.3 Траектории $R(t)$ для различных параметров

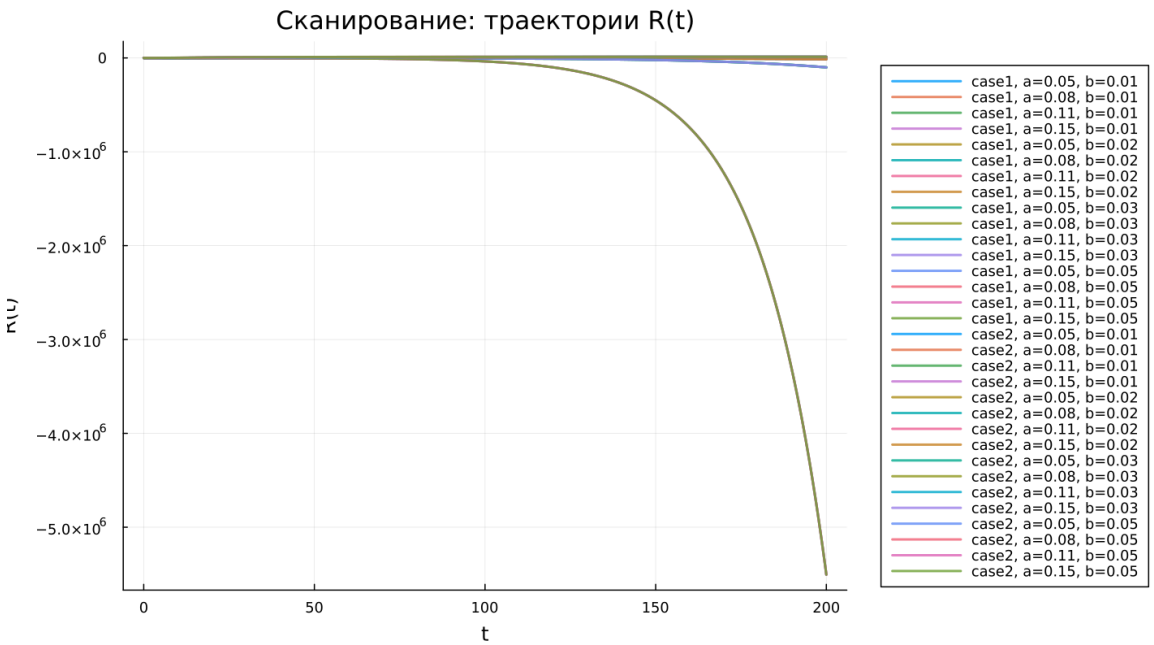

Рисунок 5.7: Сканирование траекторий R(t)

Для первой модели наблюдается убывание $R(t)$, причём значения становятся отрицательными и по модулю быстро возрастают. Это отражает некорректность модели и отсутствие физического смысла переменной.

Во второй модели, напротив, $R(t)$ монотонно возрастает и стремится к некоторому предельному значению. Параметры влияют на скорость роста: при более интенсивном заражении процесс накопления выбывших происходит быстрее.

Следовательно:

- первая модель ведёт к нефизичным значениям $R(t)$;
- вторая модель демонстрирует накопление переболевших;
- параметры влияют на скорость насыщения.

5.23.4 Фазовые траектории для различных параметров

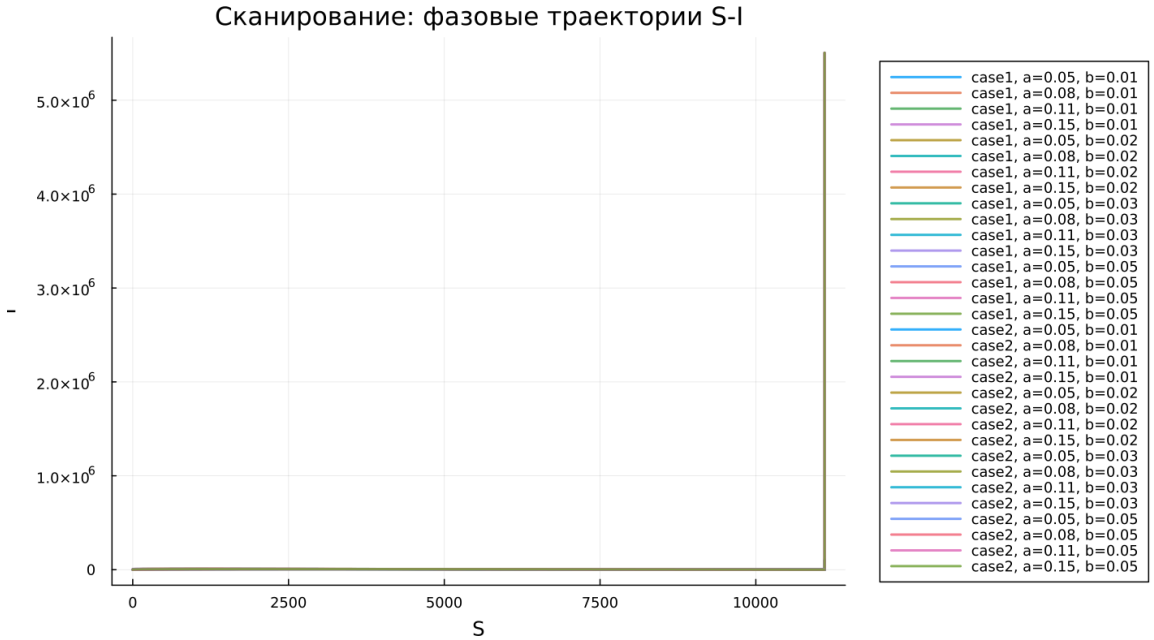

Рисунок 5.8: Сканирование фазовых траекторий

Фазовые портреты демонстрируют принципиальное различие между моделями.

Для первой модели все траектории вырождаются в вертикальные линии, поскольку S не изменяется. Это означает отсутствие полноценной двумерной динамики.

Для второй модели фазовые траектории имеют характерный вид кривых, отражающих эпидемический процесс: сначала рост I при уменьшении S, затем спад I при дальнейшем уменьшении S.

Следовательно, параметрическое изменение не меняет качественную структуру моделей:

- первая модель остаётся вырожденной;
- вторая модель сохраняет реалистичную динамику эпидемии.

5.23.5 Анализ метрики norm_final

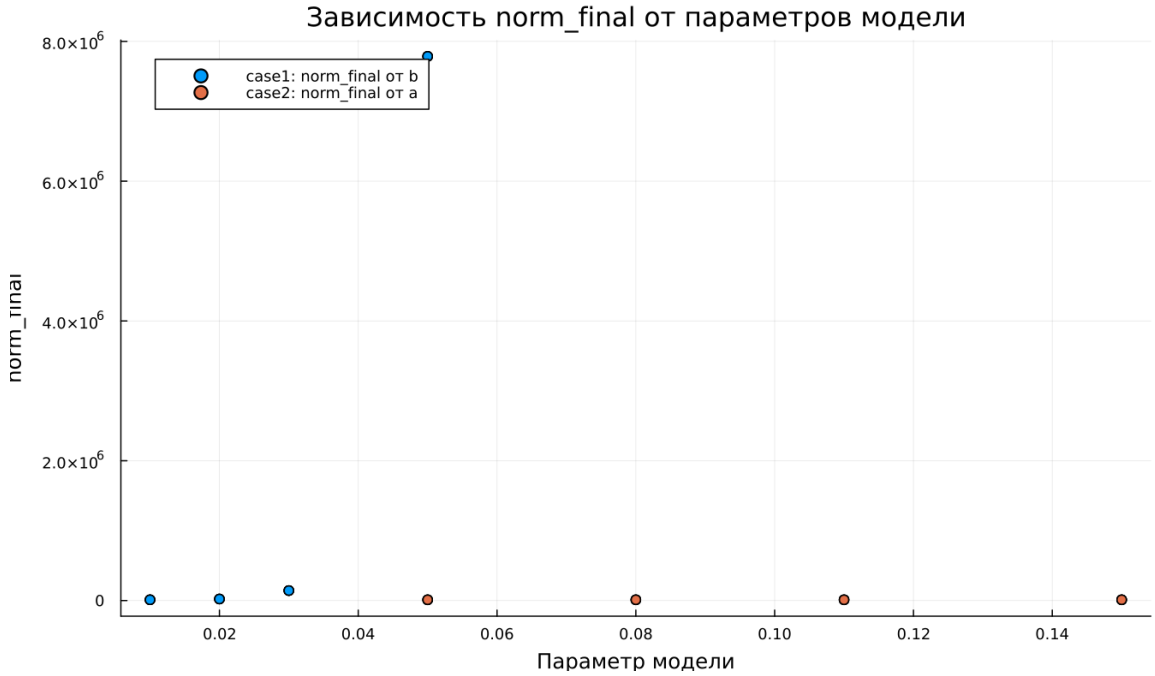

Рисунок 5.9: Зависимость norm_final

Рассматривалась метрика

$$\text{norm_final} = \sqrt{S(t_{final})^2 + I(t_{final})^2 + R(t_{final})^2}.$$

Для первой модели значение norm_final быстро возрастает при увеличении параметра b. Это связано с экспоненциальным ростом $I(t)$ и убыванием $R(t)$, что приводит к большим значениям нормы.

Для второй модели значения метрики значительно меньше и изменяются более плавно. Это объясняется тем, что система стремится к стационарному состоянию, в котором $I(t) \to 0$, а S и R принимают конечные значения.

- в первой модели метрика растёт без ограничения;
- во второй модели она отражает конечное состояние системы.

5.23.6 Анализ максимального числа инфицированных

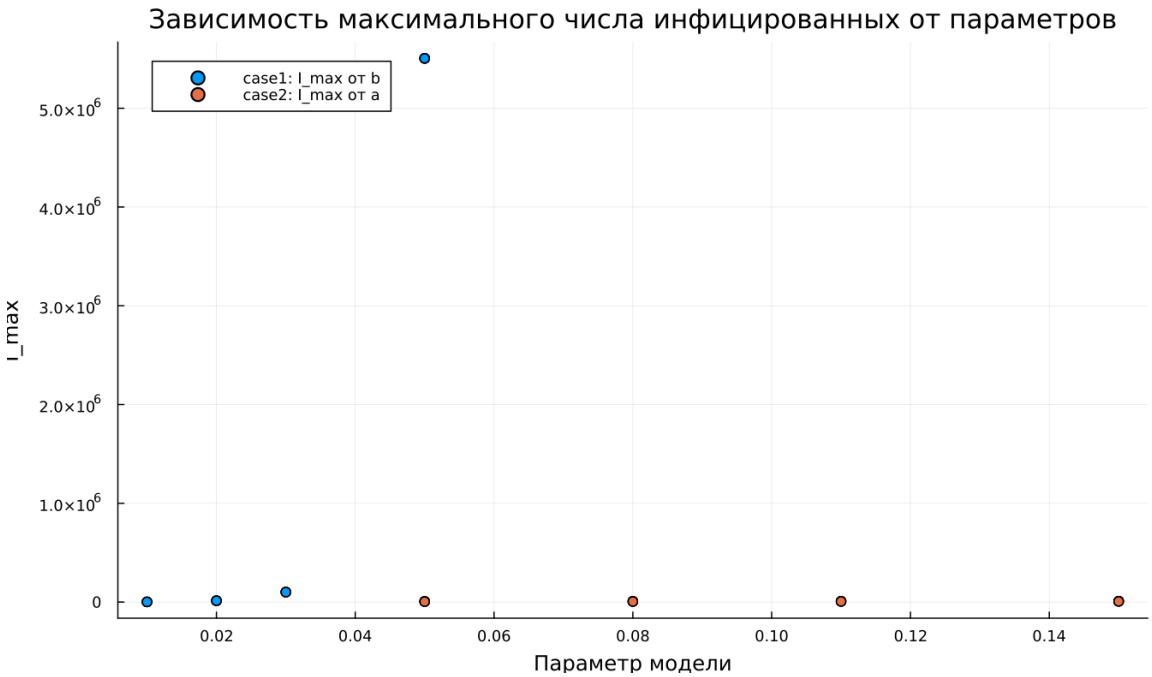

Рисунок 5.10: Зависимость I_{max}

Максимальное число инфицированных I_{max} демонстрирует сильную зависимость от параметров.

В первой модели I_{max} принимает крайне большие значения, что связано с отсутствием механизма ограничения роста. При увеличении параметра b наблюдается резкий рост максимума.

Во второй модели I_{max} имеет конечные значения и зависит от параметра a. При увеличении a пик инфицированных достигается быстрее, однако его

величина ограничена.

Следовательно:

- первая модель приводит к неограниченному росту I_{max};
- вторая модель демонстрирует контролируемую эпидемическую волну.

5.23.7 Время вычислений

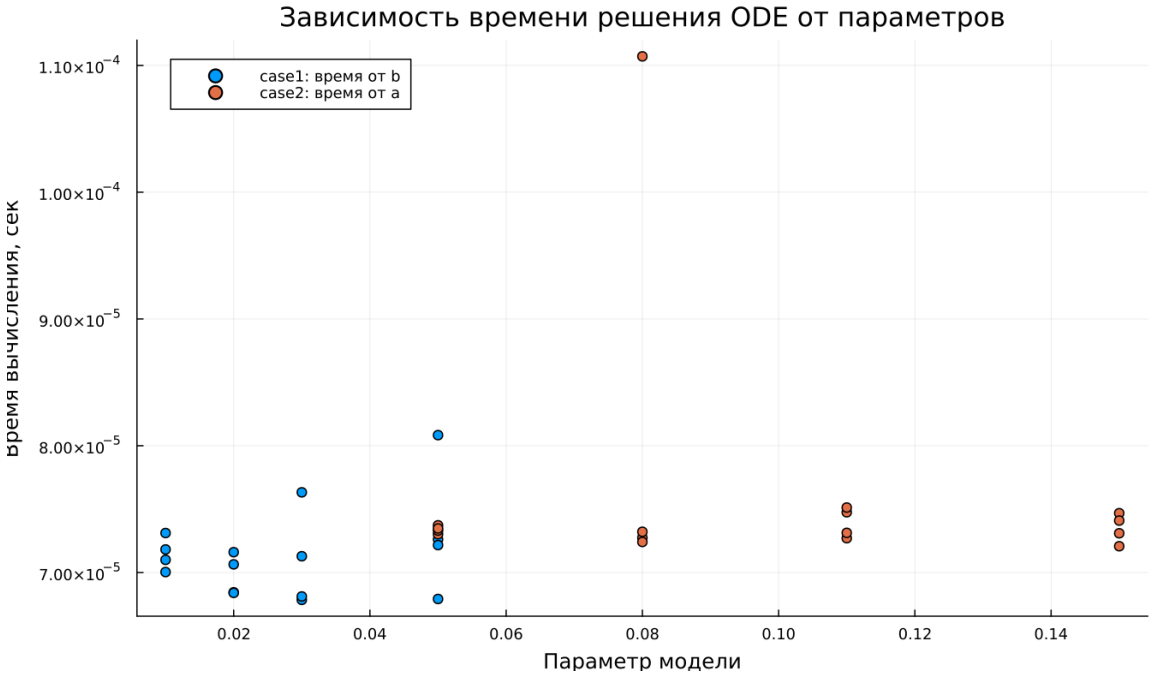

Рисунок 5.11: Зависимость времени вычисления

Бенчмаркинг показывает, что время численного решения во всех экспериментах остаётся очень малым и имеет один порядок величины ($\sim 10^{-4}$ сек).

Для обеих моделей изменение параметров оказывает незначительное влияние на время вычисления. Небольшие колебания времени связаны с особенностями численного метода и адаптивного шага интегрирования.

Можно сделать вывод, что:

- обе модели эффективно решаются численно;

- изменение параметров практически не влияет на вычислительную сложность;
- даже при экспоненциальном росте решений (case1) метод остаётся устойчивым по времени вычисления.

5.24 Выводы

1. Первая модель (case1) демонстрирует неограниченный экспоненциальный рост числа инфицированных при постоянном числе восприимчивых, что указывает на её нефизичность и отсутствие механизма стабилизации.
2. Вторая модель (case2) описывает типичную эпидемическую динамику: рост числа инфицированных с последующим спадом и переходом системы к стационарному состоянию.
3. Фазовые портреты подтверждают различие моделей: в первой модели наблюдается вырожденная динамика (вертикальная траектория), во второй — полноценная кривая, отражающая эпидемический процесс.
4. Параметры a и b существенно влияют на динамику системы: a определяет скорость уменьшения восприимчивых, а b — интенсивность роста числа инфицированных.
5. Метрика norm_final позволяет количественно различить модели: в первой она неограниченно возрастает, во второй — стабилизируется и отражает конечное состояние системы.
6. Максимальное число инфицированных I_{max} в первой модели растёт без ограничения, тогда как во второй остаётся конечным и зависит от параметров.
7. Численное решение обеих моделей выполняется эффективно, а изменение параметров практически не влияет на время вычислений.

Список литературы

1. Конструирование эпидемиологических моделей
2. Зараза, гостья наша